

WayPoint - Team ClapTac

Architecture, Data Model and AI Tool Disclosure

Submission documentation

This document explains how Waypoint is structured, how the system stores and connects operational data, and how AI tools supported the project. It gives reviewers a concise technical view of the submitted system and a transparent record of AI-assisted work.

Section	Purpose
1 System architecture	Applications, services, infrastructure and operational flow
2 Data model	Core entities, relationships and schema ownership
3 AI tool disclosure	Assisted work, human responsibilities, safeguards and limitations

1 System architecture

Waypoint is a completed four-role delivery planning and operations platform. Store managers place and track line-level orders, dispatchers create and control delivery plans, loaders prepare vehicles, and drivers execute routes with offline support, checkout, live updates and delivery evidence. Planning feasibility remains deterministic; the Go orchestrator and Python assistant service provide optional, permission-scoped AI support.

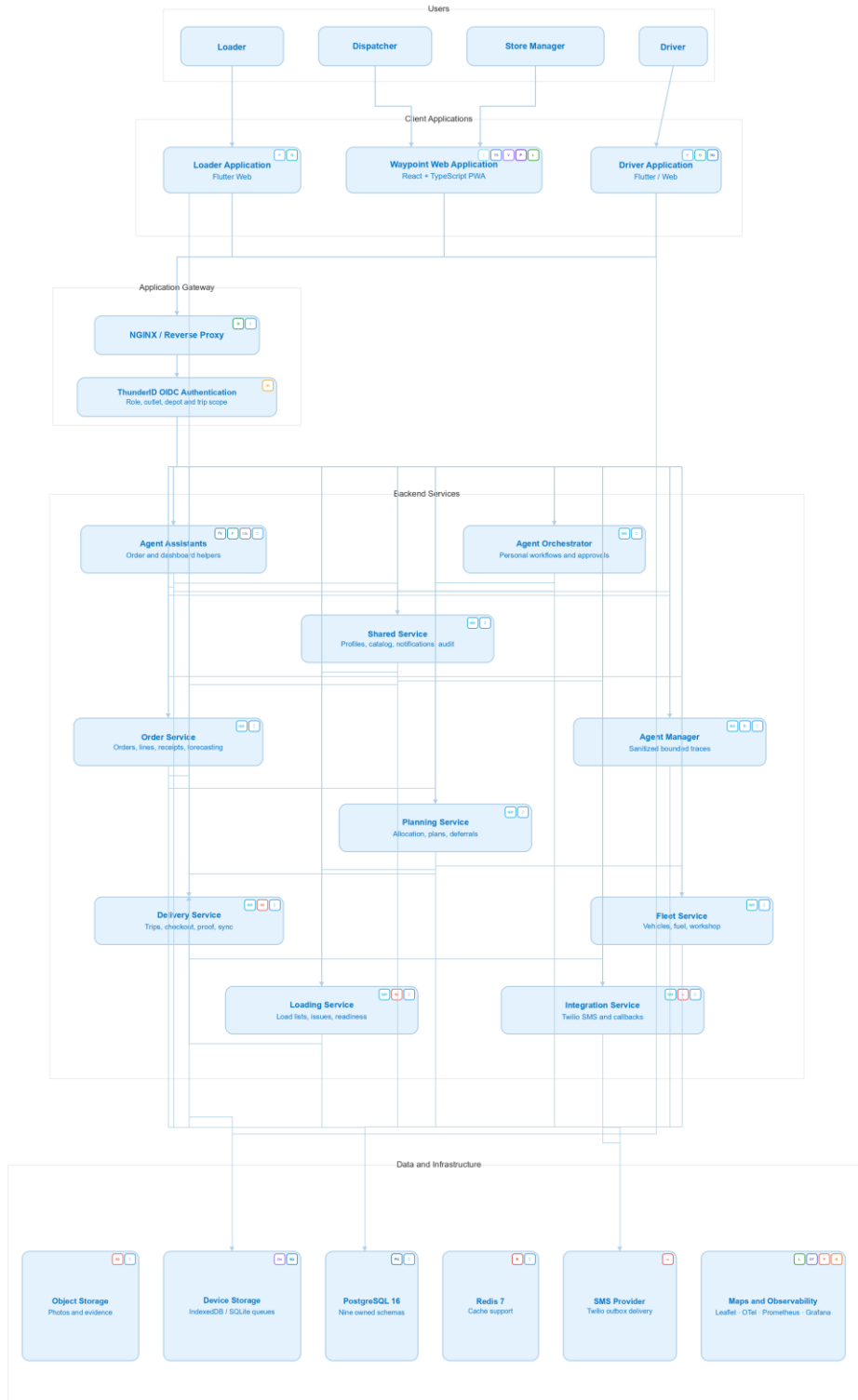


Figure 1 Main Waypoint applications, services and infrastructure

Deployment paths

Mode	Request path
Local Compose	Browser or Flutter client → NGINX → versioned Go service API
Kubernetes target	Public TLS edge and NGINX → kGateway → versioned Go service API

NGINX serves the web applications and routes API traffic. The Kubernetes target adds kGateway for domain routing, JWT and API policies, rate policies and telemetry. ThunderID supplies OIDC identity in the local environment.

Application and service responsibilities

Component	Responsibility
React web and PWA	Store-manager, dispatcher and browser driver workflows, including planning, live operations, tracking and receipts.
Flutter loader web	Versioned load lists, scanning, loading issues, chilled checks, readiness, inactivity sign-out and shared-device switching.
Driver mobile project	Offline-first route, stop, proof, issue and synchronization flows.
Order Service	Orders, cutoff rules, tracking, receipts, forecasts, reattempts and custody records.
Planning Service	Allocation, constraints, versions, publication, deferrals, disruption risks, acknowledgements and recovery.
Fleet Service	Vehicles, availability, capacity, refrigeration, fuel, incidents and workshop state.
Loading Service	Load sessions, load records, shortfalls, damage, dock checks and departure readiness.
Delivery Service	Runs, stops, checkout, outcomes, proof, messages, temperature, location and offline synchronization.
Shared Service	Profiles, scopes, outlets, calendars, policies, notifications, audit history and dashboards.
Integration Service	External notification delivery, including Twilio SMS and status callbacks.
Agent Orchestrator	Allowlisted AI tools that call authorized service APIs without direct database access.
Agent Assistants	Python 3.12, FastAPI and LangGraph order/dashboard assistants using caller-scoped service APIs.
Agent Manager	Optional Go trace collector and viewer with a bounded file-backed store.
Docker Compose	Builds and runs PostgreSQL, migrations, seed/bootstrap jobs, identity, apps and services as one local stack.

Security and service boundaries

Business APIs validate the OIDC JWT, resolve the subject through the shared user profile, enforce the required role permission, verify resource ownership and apply domain rules. Services own logical PostgreSQL schemas and communicate with peers through authenticated HTTP APIs. Cross-schema SQL is prohibited, and the agent orchestrator has no database connection.

Storage and operational infrastructure

- PostgreSQL stores structured operational records in service-owned schemas.
- MinIO-compatible object storage holds proof and loading-issue media; database rows retain references and metadata.
- Redis is available as shared cache infrastructure.
- Driver clients retain queued offline operations and cached trip data locally.
- OpenTelemetry Collector, Prometheus and Grafana provide optional telemetry and dashboards.
- OpenSearch is an optional search profile rather than a required core data store.

2 Data model

Waypoint uses PostgreSQL 16 with logical schema ownership across shared, audit, orders, planning, fleet, loading, delivery, inventory and AI schemas. Cross-service identifiers are opaque references, and services fetch authoritative data through authenticated APIs rather than joining another service’s tables.

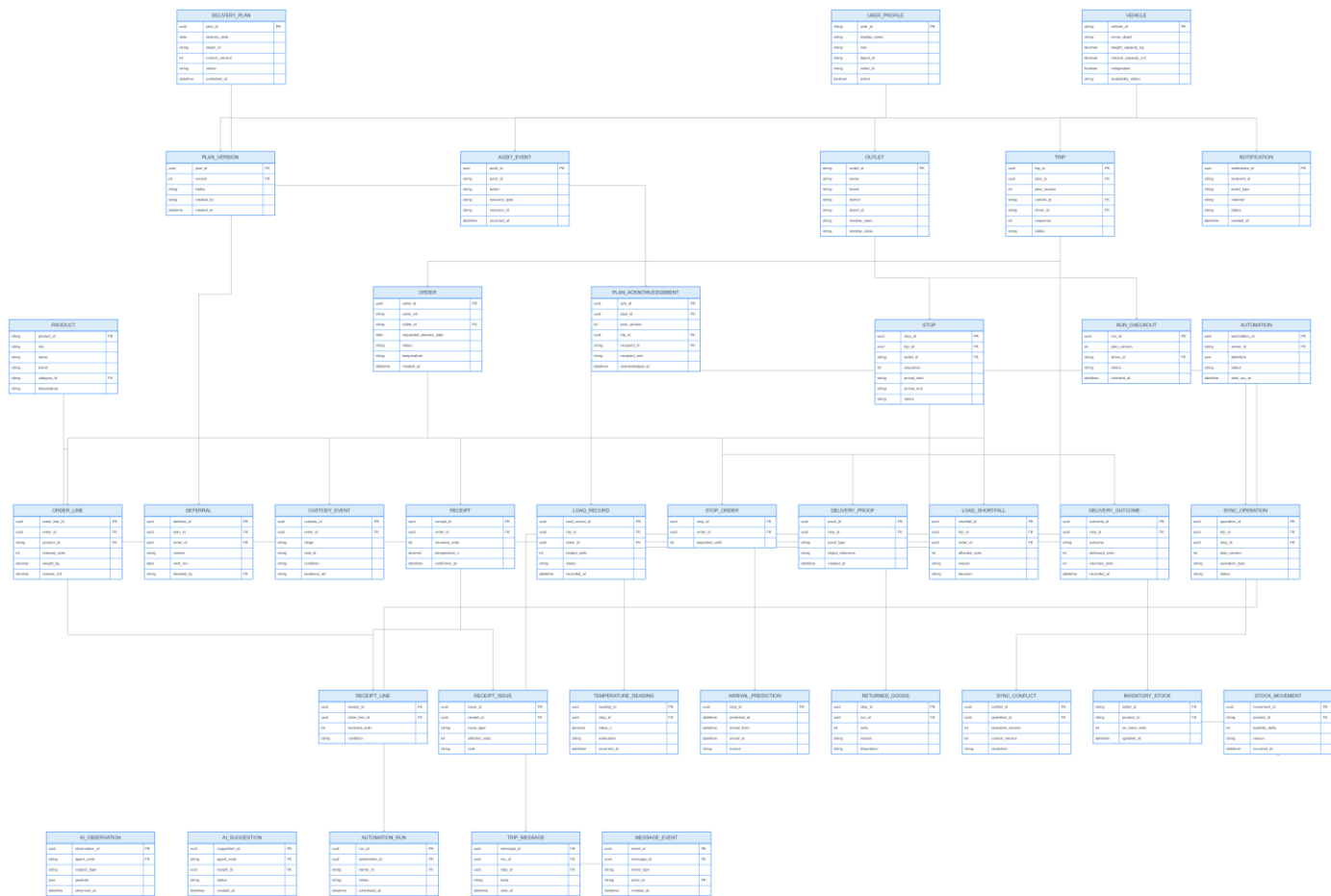


Figure 2 Service-owned schemas, supporting records and the main operational chain

Schema ownership and principal records

Schema	Principal records
shared and audit	Users and role profiles; outlets and locations; product catalog, aliases and prices; calendar and policy versions; dashboards, automations, notification outbox and audit events.
orders	Orders and order lines; import keys; receipts and receipt lines; receipt issues; forecasts and Tech custody events.
planning	Plans, plan trips, allocations, deferrals, next-run choices, publications, acknowledgements, unallocated reasons, disruption risks and immutable overrides.
fleet	Vehicles, availability, fuel entries, vehicle incidents and audit outbox records.
loading	Loading sessions, order-load rows, issues, dispatcher decisions, plan-version snapshots, dock checks and alerts.

delivery	Runs, stops, checkout, outcomes, proofs, retention, temperatures, messages, incidents, sync operations, arrival predictions and returned goods.
object and device storage	Proof photos, issue photos, signatures, object references, offline operations, cached trip data and upload state.
inventory	Simulation runs, stock levels and batches, sales and sale lines, append-only stock movements and stock counts.
ai	Agents, preferences, append-only observations, insights, suggestions, responses and agent runs.
device, object and trace storage	IndexedDB/Dexie and SQLite queues, cached trips, S3-compatible evidence, and bounded sanitized agent traces.

Main relationships

Relationship	Meaning
User → role profile → scope	A user receives a store, driver, loader or dispatcher profile that limits accessible outlets, depots and trips.
Outlet → order → order line	An outlet places an order; lines identify products, quantities, weight, volume and cooling requirements.
Plan → trip → allocation	A dated depot plan groups vehicle trips and assigns each order to a trip and stop sequence.
Plan → publication → acknowledgement	Each published plan version is immutable and field acknowledgement is tied to the relevant recipient and trip.
Trip/run → stop → outcome	The delivery run contains ordered stops; each completed stop records delivered/returned units and an outcome.
Stop → proof and temperature	Evidence and cold-chain readings are attached to the stop, with binary files held in object storage.
Order → receipt → receipt issue	The store confirms received units and records missing, damaged or mismatched quantities.
Order/plan → deferral	A deferral preserves reason, actor, decision time and required next run for fairness and store communication.
Trip message → message event	A dispatcher message can have sent and acknowledgement events.
Offline operation → conflict	A field operation retains its plan version; a mismatch can produce a durable dispatcher conflict record.
Outlet → preference → outbox	Notification preferences control external delivery while in-app operational records remain available.
Resource → audit event	Important mutations emit append-only events containing actor, action, resource and time.

Versioning, evidence and history

Plan publications, acknowledgements, master-data versions and planning-policy versions prevent silent replacement of operational decisions. Evidence metadata links object-storage files to stops, loading issues or custody stages. Proof-retention events record approved deletion activity. Audit events, custody events, stock movements and disruption overrides are treated as operational history and protected from ordinary alteration.

3 AI tool disclosure

Generative AI tools were used as development assistants during parts of the design, implementation, review and documentation of Waypoint. They did not approve releases, access production data, make real operational decisions or replace human review.

Tools used

- OpenAI Codex for repository analysis, implementation assistance, debugging, documentation and pull-request review.
- AI coding assistance for initial code and test drafts.
- Waypoint assistant components during development and demonstrations of structured order and dashboard workflows.

AI-assisted work

Area	How AI assisted
Code analysis	Repository navigation, authentication tracing, frontend/backend comparison, race-condition analysis, offline-sync review and cross-PR inspection.
Implementation	Drafting or refining session handling, field workflows, acknowledgements, deferrals, shortfalls, receipts, predictions, recovery, APIs, migrations and localization.
Testing	Suggesting or drafting checks for authorization, offline queues, plan versions, discrepancies, notification behavior and append-only rules.
Pull-request review	Identifying correctness, authorization, data-loss, production-wiring, incomplete-flow and integration issues.
Documentation	Architecture explanations, data-model summaries, feature reviews, PR descriptions and this disclosure.

All AI-generated code, tests and documentation were treated as drafts. The development team reviewed and controlled accepted changes.

Work retained by the human team

- Selecting the problem, users, requirements and submission scope.
- Making final product and architecture decisions.
- Accepting, modifying or rejecting AI suggestions.
- Reviewing and merging code.
- Managing repository access, credentials and deployment configuration.
- Conducting user acceptance and final submission review.
- Taking responsibility for the submitted system.

Use of project data

AI-assisted development used repository code, development configuration, synthetic records and test data. The tools were not intentionally supplied with production customer data, real authentication secrets, private signing keys, production database credentials or personal delivery evidence. Secrets remain outside source control and are supplied through deployment configuration.

Human review and validation

1. Review the proposed change.
2. Compare it with the functionality requirements.
3. Check authorization and data scope.
4. Inspect migrations and API compatibility.
5. Resolve integration conflicts.

6. Run relevant automated checks.
7. Validate the user-visible workflow.
8. Approve the final merge.

AI inside Waypoint

Waypoint contains structured-order, speech-assisted order capture, dashboard assistance, personal automations and traceable agent execution. The user remains authenticated, service permissions are enforced, generated structures are validated, consequential actions require user review, deterministic fallback behavior is available, and automated execution rechecks current authorization. The Go orchestrator and Python FastAPI/LangGraph assistants call allowlisted service APIs without direct database access; the optional agent manager records bounded, sanitized execution traces.

Limitations and responsibility

AI output can be incomplete or incorrect. It is not treated as an authoritative source for business rules, security decisions, operational records or release approval. Final responsibility for the design, implementation, testing, documentation and submission remains with the human project team.